
SUSTAINABLE STYLE: ENVIRONMENTAL ANALYSIS OF FASHION-MNIST MODELS VIA THE PARETO FRONTIER

Dylan Middendorf
Department of Computer Science
Miami University
Oxford, OH 45056
middendm@miamioh.edu

Jack Brandabur
Department of Computer Science
Miami University
Oxford, OH 45056
brandajj@miamioh.edu

Ryan Hardek
Department of Computer Science
Miami University
Oxford, OH 45056
hardekrm@miamioh.edu

April 28, 2026

ABSTRACT

Fashion classification has traditionally served as a benchmark task for evaluating model robustness within the computer vision community. However, dedicated research efforts in this domain have been limited. This work addresses this gap by proposing a novel convolutional neural network (CNN) architecture specifically designed for fashion classification. The proposed architecture prioritizes a balance between classification accuracy and environmental impact during model training. Prior research has documented significant carbon footprints associated with training state-of-the-art (SOTA) fashion classification models, often with minimal gains in performance. This study introduces a CNN that achieves a 34x reduction in carbon footprint compared to leading SOTA models, with a measured accuracy decrease of only 4%. The complete source code for this project is publicly available on the project's GitHub repository.

Keywords Fashion-MNIST · Image Classification · Convolutional Neural Network (CNN) · Machine Learning Models

1 Introduction

1.1 Problem Description

Image classification concerns the automated assignment of labels or tags to images, leveraging pre-existing training data. This well-established research area has witnessed significant progress over the past several decades. A comprehensive survey by Lu and Weng [48] acknowledges the inherent complexity of image classification, highlighting the absence of a single, universally effective solution. Consequently, researchers have proposed many frameworks and techniques to enhance the predictive capabilities of SOTA image classifiers. More recent trends emphasize the utilization of publicly available, community-accepted benchmark datasets. This shift stems from the potential for bias within custom-built datasets and the cost-efficiency such established datasets offer for validating novel architectures [61]. The spectrum of datasets encompasses general-purpose options like CIFAR-10 [39] and ImageNet [17], alongside domain-specific offerings such as Food-101 [10] and Fashion-MNIST [68]. This work aims to contribute to the advancement of the fashion classification landscape. To this end, we leverage the Fashion-MNIST dataset to explore the intricacies of this multifaceted domain.

The Fashion-MNIST dataset, introduced by Xiao et al. in late 2017 [68], serves as a drop-in replacement for the well-known MNIST Database of Handwritten Digits [41]. Both datasets adhere to a closed-world assumption, where all data points belong to a predefined set of categories. In the context of Fashion-MNIST, each image belongs to one of ten mutually exclusive categories: t-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, or ankle boot. Each image is represented in a 28x28 pixel grayscale format, with pixel values ranging from 0 (black) to 255 (white). The dataset comprises 70,000 images, typically split into a training set of 60,000 images and a test set of 10,000 images for model evaluation. Due to its widespread adoption, the Fashion-MNIST dataset is readily available on numerous platforms and libraries, including GitHub, Kaggle, TensorFlow, Keras, Torch, and PyTorch.

To visualize the diversity within Fashion-MNIST, we present four representative images in Figure 1. Notably, all images within the dataset share a common black background and adhere to specific standardization techniques, which will be discussed in detail in Section 6.4: Limitations. This standardization, including aggressive image scaling, ensures consistent input dimensions during data processing and reduces the computational burden for subsequent inference stages in machine learning models. However, it is crucial to acknowledge the inherent complexity of image classification tasks using the Fashion-MNIST dataset.

1.2 Importance of the Problem

The textile and fashion industries have witnessed a growing dependence on accurate image classification techniques, particularly with the rise of fast fashion [13]. This dependence fuels the development of fashion recommendation systems, which leverage user data and current trends to curate and deliver relevant product content to platform users. These systems often rely on machine learning algorithms to automate the image classification process, making the accuracy and efficiency of such algorithms paramount to the effectiveness of the entire recommendation system. Consequently, advancements in image classification research hold significant value for the textile and fashion industries.

Beyond recommendation systems, image classification plays a crucial role in trend analysis and forecasting within the fashion industry. This analysis examines various data sources, such as consumer preferences, social media trends, and runway shows, to identify popular styles, colors, and patterns [56]. By leveraging these insights from image classification, companies can forecast future fashion trends and make informed decisions regarding design and marketing strategies [58].

Image classification and other machine learning algorithms can extend their impact beyond trend analysis and forecasting to optimize inventory management within the textile and fashion industries. This optimization can be achieved through integration with Enterprise Resource Planning (ERP) systems [52]. By leveraging historical data and classification capabilities, these systems can automate inventory categorization and organization, ultimately reducing understocking and overstocking scenarios. This streamlines information flow across the organization and addresses a critical challenge in the information age: the ability to effectively utilize vast datasets for building targeted machine learning models.

1.3 Challenges

Fashion classification presents a unique challenge within the broader domain of image classification due to the inherent variability of the data. Clothing items exhibit significant variations in style, color, texture, and scale, posing difficulties for model training. While published datasets like Fashion-MNIST often employ standardization techniques to address factors like background, color, and scale, these variations contribute to the need for continued research in image classification. This research is crucial for developing robust models that can effectively balance bias-variance trade-offs and identify generalizable features within the training data.

Beyond the inherent variability of fashion data, data complexity presents another significant challenge in image classification tasks. While variability refers to the individual characteristics of each image (e.g., style, color), complexity analyzes the relationships between them. As illustrated in Figure 2, the curated Fashion-MNIST dataset exhibits noise and overlapping classes, contributing to this complexity. Consequently, simple linear classification models often struggle to achieve SOTA performance on such datasets, as evidenced in research by Xiao et al. [68]. This challenge necessitates the exploration of more comprehensive models for effective fashion classification. Recent research trends showcase advancements in data augmentation techniques during preprocessing and the utilization of artificial neural networks for classification tasks. To help visualize the inherent complexity within fashion classification, several techniques, such as principal component analysis (PCA) [1] can further highlight this complexity.

PCA, a common dimensionality reduction technique in machine learning, can be employed to visualize the complexity of datasets like Fashion-MNIST. PCA transforms high-dimensional data into a lower-dimensional, typically Cartesian, coordinate system through linear transformations. It's important to acknowledge that this process inherently discards some information, making PCA primarily a visualization tool. This technique has been used effectively in code authorship attribution (CAA) to visualize features extracted from training corpora [2, 3]. However, due to the high-dimensional nature of image data represented as numerical features, PCA is also readily applicable to image classification tasks.

To illustrate this point, we applied PCA to 2,000 images from the Fashion-MNIST dataset sample. The results, presented in Figure 2, include the data points and their corresponding class labels. Each point in the diagram represents a single image, with its color indicating its class. While PCA reveals some simplified relationships between classes, particularly between trousers, dresses, and t-shirts/tops, a significant amount of noise is also evident within the sample data.

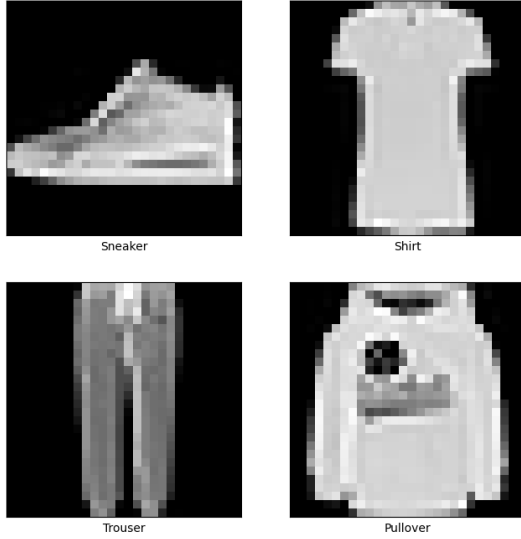


Figure 1: Four items of clothing from the Fashion-MNIST dataset. The top left image shows a pullover, the top right image shows a sneaker, the bottom left image shows a pair of trousers, and the bottom right image shows a shirt.

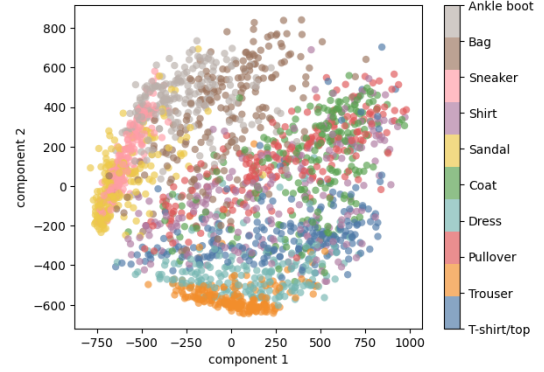


Figure 2: PCA visualization of the Fashion-MNIST dataset. This dimensionality reduction technique highlights the underlying structure of the data.

Capturing generalizable features from training data remains a significant challenge in fashion classification. Following the principle of Occam’s razor, which emphasizes the principle of parsimony, researchers have explored various techniques to address this challenge [8]. These techniques aim to strike a balance between bias and variance in machine learning algorithms, ultimately developing more robust models. This focus on balancing bias-variance is particularly crucial in fashion classification due to the inherent variability and complexity of modern datasets, as discussed previously.

1.4 Project Objectives

This study investigates the capabilities of machine learning models for fashion classification using the Fashion-MNIST dataset. We have identified four key research objectives:

1. We will implement and evaluate seven machine learning algorithms: k-nearest neighbors (KNNs), support vector machines (SVMs), Gaussian Naive Bayes (GNB), random forests, AdaBoost, Gradient Boost, and fully connected artificial neural networks (FCANNs).
2. We aim to develop a SOTA CNN that achieves performance comparable to the top published models, while significantly reducing the environmental impact associated with model training.
3. We will evaluate the performance of all developed models and compare their predictive capabilities to current SOTA models on the Fashion-MNIST dataset.
4. We will investigate the environmental impact of the trained models and explore potential strategies for mitigating this impact.

Through conducting performance evaluations across the seven different machine learning models, our comparisons revealed that Gradient Boosting was able to outperform the other models on the dataset and achieve an accuracy of over 89% due to its robustness to overfitting. Several other models achieved respectable results between 85% and 88%, including KNN, SVMs, and Random Forests. With the development of our CNN, we were able to achieve an accuracy rate of 92%, comparable to many of the current SOTA CNNs. Furthermore, we were able to reduce the environmental impact of the model by over 34 times, while only having a 4% lower accuracy than the respective models. While architectures, such as AmoebaNet-B + cutout, emit over 9,000 kgCO₂ eq., the CNN we created has a significantly lower carbon footprint of 0.0858% kgCO₂ eq. This presents the need for mitigation strategies that can continue reducing the environmental impact. We also implemented lifecycle management techniques by using the codecarbon library to document energy consumption, which supports regular model evaluation for environmental impact. Through using these strategies, we can continue to optimize the carbon impact.

2 Literature Review

2.1 Introduction

This section reviews the research that has been completed on the Fashion-MNIST dataset using complex algorithms, tweaking hyperparameters, and employing different activation functions. Furthermore, it dives into the SOTA models that are being employed currently to achieve a 96.91% accuracy. We then discuss the gaps and limitations of the prior work due to the generalized use of these models over many datasets. Finally, we talk about the prior work we will build on as well as explore the computation resources required for training these state-of-the-art models.

2.2 Prior Works on Fashion-MNIST

The recently released Fashion-MNIST dataset (2017) has already established itself as a valuable resource for benchmarking and advancing machine learning, particularly in the field of deep learning. Due to its compatibility with the widely used MNIST dataset, Fashion-MNIST facilitated research even before its official publication. For example, Liao and Couillet [44] leveraged Fashion-MNIST to introduce least squares support vector machines (LS-SVM). Similarly, Kolchinsky et al. [38] employed the dataset to propose a technique for utilizing nonlinear information bottlenecks within neural networks. Notably, Agarap's [4] seminal work on rectified linear units (ReLU) is credited with bringing Fashion-MNIST to wider attention within the machine learning community. These early studies, along with numerous subsequent works, have solidified Fashion-MNIST's position as a well-established benchmark within the field.

Recent research on Fashion-MNIST image classification has primarily focused on enhancing the performance of convolutional neural networks (CNNs) through regularization and hyperparameter tuning techniques. These studies often prioritize developing generalizable architectures and techniques that broadly improve the overall predictive performance of CNNs for image classification tasks, while achieving SOTA accuracy on Fashion-MNIST serves as a benchmark. Here's a summary of some key advancements:

- Zhong et al. [71] introduced random erasing, a data augmentation method that artificially expands dataset diversity, by randomly replacing a region within an image with noise, essentially "erasing" it. This approach achieved a SOTA accuracy of 96% using the WRN-28-10 architecture [70].
- Harris et al. [30] further improved SoTA performance to 96.36% by incorporating FMix, a data augmentation technique that combines elements from two images within the same class, into a PreAct-ResNet18 model [33]. This technique helps the model focus on crucial image regions.
- Tanveer et al. [60] surpassed this accuracy by employing a fine-tuned DARTS architecture with advanced data augmentation and rigorous hyperparameter tuning via a gradient-based search. This approach achieved the current highest reported performance on the dataset, with a SoTA accuracy of 96.91%.

Figure 3 provides a detailed overview of the SoTA model's accuracy concerning publication date, visually illustrating the increase in predictive performance over time.

While achieving SOTA accuracy on Fashion-MNIST serves as a benchmark, it is crucial to recognize that the primary emphasis of much of this research lies in developing generalizable architectures and techniques. These advancements aim to broadly enhance the overall predictive performance of CNNs for image classification tasks. Consequently, many of the proposed models possess millions of trainable parameters, enabling them to leverage diverse data augmentation methods and achieve marginally superior SOTA performance on datasets like Fashion-MNIST. However, these models are not tailored specifically for fashion classification, but rather for the broader domain of image classification.

2.3 Novel Approaches and Contributions

This study builds upon the foundational work of Kadam et al. [36] who investigated the influence of hyperparameter tuning on the performance of CNNs. Their research presented five distinct CNN architectures with a progressively increasing number of convolutional layers. Architecture 1 consisted solely of a classifier (two fully connected layers) without any internal regularization techniques (e.g., dropout layers). Architecture 2 incorporated two convolutional layers with a 2x2 kernel followed by the classifier. Architectures 3, 4, and 5 progressively added convolutional layers (3, 4, and 4 respectively) with Architecture 5 also utilizing a larger kernel size (3x3). Kadam et al. employed these architectures to systematically evaluate the impact of various hyperparameters, including batch size, learning rate, and optimizer selection, on the models' predictive performance.

Kadam et al. achieved impressive results on the Fashion-MNIST dataset using their proposed architectures. Notably, Architectures 3 and 5 attained the highest testing accuracy of 93.56%. Both architectures leveraged a batch size of 128,



Figure 3: This figure displays the percentage error of various algorithms when used for image classification on Fashion-MNIST, highlighting the algorithms with the lowest percentage error.

the softmax activation function, and dropout fixed at 25% for regularization. The models were trained using the Adam optimizer [37] with a learning rate of 0.002 for 50 epochs. While these accuracies approach the aforementioned SOTA models, they exhibit a slight performance degradation.

Despite the encouraging results achieved by Kadam et al., their work presents some limitations that this study aims to address in Section 5: Results. Notably, the authors did not include key visualizations for model analysis, such as confusion matrices, which are instrumental in identifying areas for improvement in classifier design and performance [28]. Furthermore, the study lacks model validation techniques, such as k-fold cross-validation, which are essential for ensuring model robustness [67]. Without validation, models can overfit the specific characteristics of the test set, leading to overly optimistic performance.

2.4 Environmental Impacts of Model Training

The growing complexity of artificial intelligence models has led to a significant increase in the number of trainable parameters. This trend necessitates a corresponding rise in computational resources during model training. For instance, the Transformer architecture [63] typically comprises billions of parameters due to their stacked encoder and decoder modules. Similarly, deep residual networks, such as ResNet-101, ResNet-152, and ResNet-1001 [32, 33], require substantial computational resources for training due to their hundreds or even thousands of layers.

Beyond these large models, grid search, a hyperparameter optimization technique, has become increasingly common [34]. Grid search exhaustively trains the model across all specified hyperparameter combinations to identify the optimal configuration for a given dataset. This exhaustive approach can consume vast amounts of computational resources and, consequently, energy, as numerous model iterations are executed to find the best-performing hyperparameters. This ever-increasing demand for computational resources has a negative environmental impact due to the resulting CO2 emissions.

Several strategies can be employed to mitigate the environmental impact of machine learning training. Model regularization techniques, such as L_1 regularization, can be leveraged to eliminate weights that do not contribute to improved model performance, thereby reducing model complexity [65]. Additionally, model compression techniques like quantization can be utilized to further reduce the environmental footprint [45]. Quantization allows models to exploit varying bit widths within floating-point calculations, significantly reducing memory requirements for storing

model weights. Beyond model architecture considerations, efficient hyperparameter optimization techniques can be employed to minimize training times. Random search, for example, has been shown to achieve comparable or even superior performance to grid search while requiring fewer training runs [6]. The implementation of these techniques in the context of our proposed model will be further explored in Section 5: *Experiments and Results*.

3 Background

3.1 Introduction

Machine Learning has changed the way we solve advanced problems in every industry ranging from finance to healthcare. In this section, we dive into the background of machine learning, where we touch on its capabilities and several subfields, including supervised, unsupervised, and semi-supervised learning. Then, we explore 9 key methodologies by explaining how they work, implementation strategies, and the advantages/disadvantages they hold. These algorithms continue to be applied in Section 5: *Experiments and Results*, where they are applied to the Fashion-MNIST dataset.

3.2 Introduction to Machine Learning

Machine learning, a subfield of artificial intelligence, has been a key area of research for decades. It has demonstrably reduced development complexity in historically challenging domains like computer vision and natural language processing. Its broad applicability extends to healthcare [55], finance [27], marketing [64], and robotics [11], where problems are often difficult to model and generalize. Machine learning offers an algorithmic approach to uncover patterns in numerical data and transform this feature space into practical solutions. This capability significantly reduces development time compared to traditional software development techniques. Furthermore, machine learning solutions are often more abstract, allowing for adaptation to problems with similar characteristics. Consequently, machine learning has become a cornerstone for augmenting traditional development practices across diverse fields.

Machine learning can be broadly categorized into three subfields: supervised, unsupervised, and semi-supervised learning [62]. Supervised learning, the most common approach, involves training a model using labeled data, where each data point has a corresponding known outcome. This technique excels in tasks like image classification, where training data includes pre-labeled images. Since fashion classification inherently falls under image classification, this paper will focus on supervised machine-learning algorithms for this specific application. Unsupervised learning, in contrast, aims to identify patterns and structures within unlabeled data, enabling further exploration. Finally, semi-supervised learning tackles scenarios with mixed datasets, containing both labeled and unlabeled examples.

Supervised machine learning offers a diverse set of algorithms, each with trade-offs between predictive performance, implementation complexity, and computational cost. Researchers have employed various popular algorithms on the Fashion-MNIST dataset to identify the most effective solution for fashion image classification, as seen in Figure 3. This report highlights the importance of selecting an algorithm that balances these factors for the specific application.

3.3 K-Nearest Neighbors (KNNs)

KNN is an algorithm that is most often used in classification problems, although it can be used for regression problems as well [29]. KNN uses known, classified points to determine where an unclassified point may be located through the use of distance formulas and majority voting.

By taking the unclassified point and choosing between several distance metrics, we can determine which of the majority of classified points the point is nearest to, allowing us to classify the unknown point. Furthermore, a k value to find the number of neighbors needs to be chosen that optimizes the accuracy of the algorithm. Because majority voting is used, the value of k should be odd to break ties. The more outliers the data may contain, the higher a k value should be chosen [16].

In the context of machine learning, various distance metrics quantify the similarity or dissimilarity between data points. Euclidean distance, a widely used metric, calculates the straight-line distance between two points in a multidimensional space. Manhattan distance, in contrast, measures the total distance traveled along each axis when moving from one point to another. It is calculated through the sum of the absolute differences between corresponding coordinates. Notably, both Euclidean and Manhattan distances can be derived from the Minkowski distance formula (Equation 1), a general framework for distance calculation. By varying the parameter p within this formula, we can recover Euclidean distance (when $p = 2$) and Manhattan distance (when $p = 1$).

KNN offers several advantages. Its primary strength lies in its inherent simplicity. The algorithm identifies the k nearest neighbors based on a chosen distance metric and assigns the majority class label. Consequently, hyperparameter

$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad (1)$$

optimization, often involving grid search, requires a minimal number of inferences on the test set. However, this very simplicity can lead to overfitting, as KNN has limited regularization techniques available. This stems from the fact that the model effectively resides within the training data itself. Inferences over the test data can be computationally expensive, with an average time complexity of $O(n \log n)$ per data point due to distance calculations.

3.4 Support Vector Machines (SVM)

SVM is an algorithm that is used for both classification and regression. It is most often used in binary classification tasks, such as determining whether or not an email is considered spam. The idea behind SVM is to find the most optimal way to separate points of different classes with a hyperplane. To create this hyperplane, or decision boundary, support vectors, which are the points nearest to the hyperplane, need to be identified. The hyperplane equation of $w \cdot x + b = 0$ is found where w is the weight vector and b represents the bias. The support vectors use the equation $w \cdot x_+ + b = +1$ for the positive class and $w \cdot x_- + b = -1$ for the negative class. The distance from the support vectors and decision boundary has to be reduced to as much as possible. There is also a margin, which is the distance from the decision boundary to the closest points. We want to maximize this margin to allow the hyperplane to be the largest distance from the class possible, to prevent misclassification that may occur [5]. The margin is calculated using the equation $M = \frac{2}{\|w\|}$. To maximize the margin, we must minimize the value $\frac{1}{2}\|w\|^2$. This is because the norm of the weight vector must be minimized while also making sure the data points are classified the right way. Another popular concept with SVM is the kernel trick, which allows SVM to work in space that would be higher than two dimensions. The linear kernel is most often used with linearly separable data, and represented by the equation: $K(x_i, x_j) = x_i^T x_j$. You would use this kernel when there are a lot of features where overfitting may be occurring. Another type of kernel is the polynomial kernel, which is used for non-linear separable data where the classification may represent more complex space. The equation for a polynomial kernel is $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$, where γ is the scale factor, r is the offset, and d is the degree of the polynomial. It is most often used for classifying text and processing images. The third most common kernel trick is the Radial Basis Function Kernel or Gaussian kernel. The RBF equation is represented by $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ and is best used when we do not know anything about the data [14]. It is also able to be used in infinite-dimensional space.

3.5 Naive Bayes Classifier

The Naive Bayes classifier is a probabilistic approach to classification that utilizes Bayes' theorem to estimate the posterior probability of a class given a set of features (x). The core principle is captured in the equation: $P(C_k|x) = \frac{P(x|C_k)P(C_k)}{P(x)}$, where $P(C_k|x)$ represents the probability of class C_k occurring given the data x , $P(x|C_k)$ represents the probability of observing data x belonging to class C_k , $P(C_k)$ signifies the prior probability of class C_k , and $P(x)$ denotes the total likelihood of observing the data x (often omitted since it acts as a normalization constant). This method assumes conditional independence between features within a class, simplifying the computation but might not always hold in real-world scenarios. Despite this limitation, Naive Bayes offers several advantages. It is computationally efficient due to its simple structure and performs well with limited training data [66]. Different variations of Naive Bayes exist, including GNB for continuous features, Multinomial Naive Bayes for discrete features with frequencies, and Bernoulli Naive Bayes for binary features. These variations cater to specific data types, making Naive Bayes a versatile tool for various classification tasks.

3.6 Ensemble Learning

Ensemble learning is a machine learning technique that combines multiple learning algorithms to improve predictive performance and robustness by leveraging majority voting. Specifically, these ensemble methods enable a given model to express more flexibility within its structure within alternative algorithms [19]. Furthermore, ensemble learning can often produce high-performance models with low variance. Given the increasing complexity of real-world datasets and the demand for highly accurate predictions, ensemble learning serves as a powerful tool for image classification [69]. As a result, extensive research has been conducted to develop a wide selection of ensemble methods to emulate SOTA results with reduced computational overhead.

3.7 Random Forest

Random forests are a commonly used method in machine learning that uses multiple decision trees for classification and regression problems. Decision trees are created by taking the data and separating it into branches to form a tree, with the branches in the tree forming decision rules that go down to the leaf nodes, or the results. When random data samples are taken, the trees of the random forest are built, with feature randomness occurring at the split nodes. For image classification tasks such as Fashion-MNIST, the trees in a random forest vote for a class with majority voting and the class with the largest amount of votes represents the model's prediction [9].

The primary technique that random forests use is called Bagging, and it can be split up into bootstrap sampling and aggregation. Bootstrap sampling is the process of the decision trees being trained on different subsets of the data to lower the overall amount of variance. After this occurs, aggregation is applied to ensure a final prediction is created by using the predictions from the individual trees.

One of the biggest disadvantages of random forests is the computational intensity, as the large number of trees requires significant amounts of memory [7]. They are also often seen as black boxes due to the high amount of factors that are used to make the predictions. However, they are best used for large datasets where hundreds or thousands of input variables must be managed. Furthermore, because of their complexity, they oftentimes provide the highest levels of accuracy due to their robustness and ability to create trees with imbalanced data.

3.8 AdaBoost

Boosting involves sequentially training weak learners by emphasizing misclassified samples to bolster the predictive performance of the whole ensemble. This abstract idea has been implemented into several algorithms, the most popular including AdaBoost, GradientBoost, and XGBoost. Most boosting algorithms differ from their method of weighing examples, and consequently, hypotheses.

AdaBoost, an abbreviation for adaptive boosting, is a meta-estimator that utilizes weak learners to learn the characteristics of a given dataset. Specifically, the algorithm begins by fitting a single classifier to the dataset and fits subsequent classifiers to emphasize more "difficult" examples within the dataset. This technique has been utilized on a myriad of datasets and provides a considerable improvement to predictive performance. The technique was originally developed by Freund and Schapire [23]; however, the algorithm has been extensively researched, which fostered the development of improved algorithms. One such improvement, SAMME, was published by Zhu et al. [31], which provides support for multi-class classification, making it ideal for the Fashion-MNIST dataset.

As the pioneering algorithm in the field of ensemble boosting, AdaBoost adopts a clear and intuitive approach. It achieves this by sequentially constructing an ensemble of weak learners, strategically allocating greater focus to observations that were previously misclassified. In conjunction with weight adjustments for individual observations, AdaBoost assigns weights to the weak learners themselves. These weights empower more successful learners to exert a greater influence during the final prediction for unseen data. Following the training of all weak learners, the model leverages each classifier's contribution, weighted accordingly, to generate a prediction. A majority vote across the classifications from all weak learners determines the final output.

$$err^{(m)} = \frac{\sum_{i=1}^n w_i \mathbb{I}(c_i \neq T^{(m)}(x_i))}{\sum_{i=1}^n w_i} \quad (2)$$

$$\alpha^{(m)} = \ln \frac{1 - err^{(m)}}{err^{(m)}} + \log(K - 1) \quad (3)$$

The AdaBoost algorithm commences by initializing uniform weights of $1/n$ for all observations. Subsequently, a weak classifier, typically a decision tree with a single level, is fitted to the training dataset. Following this, the classification error is computed using Equation 2 where $\mathbb{I}$ denotes the indicator function. This error metric can be interpreted as the expected value of the misclassification weights across all data points. It serves as a measure for the ensemble to assess the average performance of the hypotheses generated by the preceding weak classifier. Subsequently, a coefficient, denoted by $\alpha^{(m)}$, is calculated based on the number of classes, K , and the previously computed error, $err^{(m)}$, as seen in Equation 3. This coefficient, $\alpha^{(m)}$, acts as a normalized coefficient that influences the impact of the weak learner within the model classification. It is determined by the error rate of the weak classifier; a lower error results in a higher weight for the classifier, while a higher error yields a lower weight. Following the computation of $\alpha^{(m)}$, the observation weights are updated to emphasize instances that the weak learner misclassified. This is achieved through

the formula: $w_i \leftarrow w_i \cdot \exp(\alpha^{(m)} \cdot \mathbb{I}(T^{(m)}(x_i)))$. Here, the weight of each observation, w_i , is adjusted proportionally to the indicator function, $\mathbb{I}(c_i \neq T^{(m)}(x_i))$, which takes a value of one for misclassified instances and zero otherwise. The term $\exp(\alpha^{(m)})$ acts as a scaling factor, amplifying the weights of misclassified examples and diminishing the weights of correctly classified ones.

3.9 Gradient Boosting

Gradient boosting is a powerful ensemble learning technique that builds upon the concept of boosting. Unlike AdaBoost, which is restricted to discrete loss functions, gradient boosting offers greater flexibility by allowing the implementation to leverage any differentiable loss function [50]. Common choices in this regard include mean squared error for regression tasks and log loss for classification problems. The algorithm iteratively constructs an ensemble of weak learners, typically decision trees, with each iteration focusing on minimizing the chosen loss function. This minimization is achieved through the concept of pseudo-residuals, which highlight observations within the training data that hold the most potential for improving the model's predictive performance. Notably, gradient boosting employs a weighted approach for classification tasks, where predictions from individual trees are combined based on their respective contributions to reducing the loss function.

This section focuses on the core steps involved in gradient boosting for classification, using the Iris dataset as an illustrative example. It is important to note that gradient boosting for regression shares many similarities with the classification algorithm. The classification process begins with determining the log loss of the predicted value, which can be expressed mathematically as $\text{logit}(\rho) = \ln(\rho/(1 - \rho))$. Here, ρ represents the predicted probability. Subsequently, the pseudo-residual is calculated by subtracting the actual class label from the predicted probability. The actual class labels can be obtained using the indicator function, $\mathbb{I}$, where a value of 1 denotes a true classification and 0 represents a false classification.

$$w = \frac{\sum_{i=1}^n |y_i - T^{(m)}(x_i)|}{\sum_{i=1}^n \rho_i^{(m-1)} \times (1 - \rho_i^{(m-1)})} \quad (4)$$

$$\rho = \frac{e^{-\ln C(x_i)}}{1 + e^{-\ln C(x_i)}} \quad (5)$$

Following the calculation of residuals, a weak learner is fitted to the observations. Once the weak learner is fitted, the weight for each leaf node can be determined using Equation 4. This weight is crucial for subsequent weak learners. The derivation of the weight allows for the fitting of additional weak learners. This is achieved by first obtaining the cumulative predicted probability for all observations by summing the weights from all the fitted weak learners. The final predicted probability is then obtained by inverting the cumulative log-odds (Equation 5), where $C(x_i)$ represents the predicted log-odds ratio of the classifier.

3.10 Fully Connected Artificial Neural Networks (FCANNs)

FCANNs form the cornerstone of deep learning architectures. These networks typically consist of an input layer, followed by multiple stacked hidden layers with activation functions, culminating in an output layer. Each layer is fully connected to the subsequent layer, allowing the network to learn complex, non-linear transformations of the input data through matrix multiplications and activation functions. Despite advancements in deep learning architectures, FCANNs remain prevalent due to their effectiveness in learning these non-linear relationships.

Training an FCANN involves two key steps: forward propagation and backpropagation. During forward propagation, the input data is propagated through the network layer by layer. At each layer, the data is multiplied by a weight matrix and a bias vector, followed by a non-linear activation function that introduces non-linearity into the network's decision boundaries. This process is mathematically represented by: $y = f(W_n x_n + b_n)$, where y represents the output of the layer, f represents the activation function, W_n denotes the weight matrix of the current layer n , x_n signifies the input vector to the layer, and b_n represents the bias vector for the layer.

Following forward propagation, the calculated output is compared to the ground truth label, and an error is computed using a loss function, such as mean squared error (MSE) or cross-entropy loss for multi-class classification problems. Backpropagation utilizes the chain rule to efficiently propagate the error back through the network, calculating the gradients of the loss function concerning each weight and bias parameter. These gradients are then used to update the network parameters via an optimization algorithm like gradient descent, minimizing the overall loss and improving

the network's performance in future training iterations. This process of forward propagation, error calculation, and backpropagation is repeated iteratively until the network converges to a minimum loss state.

To provide FCANNs with the capability of learning non-linear relationships within data, non-linear activation functions are incorporated within each layer. These functions introduce essential non-linearity into the network's decision boundaries, allowing it to model complex patterns. Common activation functions include sigmoid, tanh, and ReLU (Rectified Linear Unit) with its variants.

The sigmoid function, previously a mainstay in neural networks, is defined by a sigmoidal curve with an output range between 0 and 1. Another frequently used function is the hyperbolic tangent (tanh), which offers an expanded range of -1 to 1 compared to sigmoid. More recently, the ReLU function has gained prominence due to its computational efficiency during both forward and backpropagation. However, ReLU is susceptible to the "dying ReLU problem," where certain neurons might become permanently inactive during training. These three exemplar activation functions are visualized in Figure 4 for comparison.

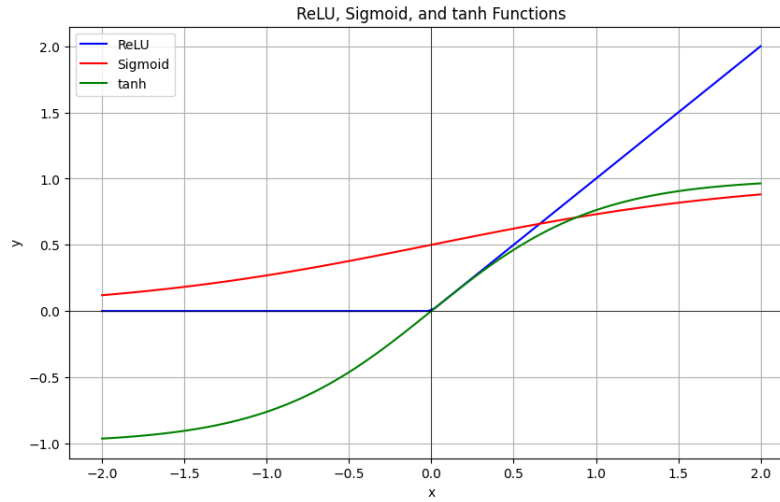


Figure 4: Activation Functions. This figure compares the rectified linear unit (ReLU), sigmoid, and hyperbolic tangent (tanh) activation functions.

While FCANNs excel at various classification and regression tasks, they exhibit inherent limitations when applied to image classification. One key challenge lies in preserving spatial information within the data. FCANNs typically process pixels independently, neglecting the crucial contextual information provided by neighboring pixels. This disregard for spatial relationships hinders the network's ability to learn and identify higher-level, abstract features within an image. Consequently, FCANNs also struggle with translation invariance. The network may struggle to recognize the same pattern in different image locations unless the training data specifically includes such variations. These limitations hinder the model's ability to generalize effectively to unseen data, impacting its overall performance in real-world image classification scenarios.

3.11 Convolutional Neural Networks (CNNs)

The theoretical foundations of CNNs can be traced back to Fukushima's seminal work [25], where the neocognitron architecture was introduced. This influential paper established two fundamental layers that are now prevalent within modern convolutional blocks: the convolutional layer and the pooling layer. The convolutional layer performs a weighted cross-correlation operation on the input, typically computed over a two-dimensional feature map. This operation is applied over a restricted region, known as a kernel, to capture local spatial information within the input. To address potential internal covariate shifts within the convolutional layer, batch normalization is often employed. This normalization technique, introduced by Ioffe and Szegedy [35], has improved the convergence properties of CNNs. Following normalization, a non-linear activation function is applied to the filtered output, enabling the model to learn non-linear relationships within the data.

A popular choice for the non-linear activation function in CNNs is the rectified linear unit (ReLU)[24]. This function introduces non-linearity by applying a threshold at zero. Mathematically, $ReLU(x) = \max(0, x)$, where x is the input value. Consequently, negative inputs are set to zero, while positive inputs are passed through unchanged. This

characteristic is depicted graphically in Figure 4. Additional convolutional layers can be employed following the activation layer to extract higher-level features. Subsequently, pooling layers are typically applied to reduce the dimensionality of the data while preserving salient information. This down-sampling strategy involves passing a kernel over the feature maps and retaining a specific value (e.g., the maximum) within the defined region. The pooling operation can significantly reduce the size of the data representation while capturing essential spatial relationships.

While convolutional layers are the cornerstone of CNN architecture, fully connected (FC) layers composed of artificial neurons are also employed. These FC layers typically follow the convolutional blocks and function as a classifier. Each artificial neuron within an FC layer represents a linear transformation, taking a weighted sum of its numerical inputs. Mathematically, this can be expressed as: $\hat{y} = \sum_{i=1}^n (w_i * x_i) + w_0$, where $\hat{y}$ is the output of the neuron, w_i represents the weight associated with the i -th input (x_i), and w_0 denotes the bias term. These neurons can be arranged sequentially or in parallel to form a complex network. In CNNs, FC layers leverage the feature maps extracted by the convolutional blocks to perform high-level reasoning and classification tasks.

The sequence of convolutional and FC layers culminates in the generation of a finite set of logits. These logits represent the raw, unnormalized predictions for each class. To convert these logits into a probability distribution suitable for multi-class classification, a softmax function is applied [26]. Mathematically, the softmax function can be expressed as:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (6)$$

Where $\sigma(\mathbf{z})$ denotes the softmax output vector, z_i represents the i -th element (logit) within the input vector $\mathbf{z}$ and K represents the total number of classes. This function effectively normalizes the logits by transforming them into a probability distribution where each element lies between 0 and 1, and the sum of all elements equals 1. This probabilistic interpretation allows the model to indicate the likelihood of an input belonging to each potential class.

4 Methods

4.1 Introduction

In this section, we detail the methodologies used in our study, starting with the preprocessing of the Fashion-MNIST dataset. We describe the conversion of the dataset to formats suitable for different models, ensuring compatibility across frameworks. Next, we document the development of our CNN model, discussing the design process tailored to the dataset’s characteristics. Advanced techniques such as data augmentation and normalization are used to enhance model performance. Lastly, we outline the training and evaluation procedures, including hyperparameter optimization and performance metrics selection.

4.2 Data Preprocessing

The Fashion-MNIST dataset is a well-established benchmark employed in various machine-learning libraries and frameworks. In this study, the dataset was retrieved from the PyTorch library [51]. The data structure was subsequently converted to suit the specific requirements of the chosen model. For instance, scikit-learn-based models typically operate on pandas DataFrames, while deep learning models in PyTorch utilize tensors. This standardized format of the Fashion-MNIST dataset minimized the need for extensive preprocessing steps.

For the CNN model, a more comprehensive data preprocessing pipeline was implemented to enhance the dataset’s diversity and bolster model performance artificially. A detailed illustration of the techniques employed for data normalization and augmentation is provided in Figure 6.

One specific augmentation technique involved a 50% chance of applying a horizontal flip to each image. This process enables the network to learn features independent of their spatial orientation, thereby mitigating any potential bias introduced by the training set. For example, if a particular pattern were predominantly located on the right side of a garment in the original images, the horizontal flip augmentation would expose the corresponding neurons responsible for detecting the pattern on the left side as well.

Furthermore, the image data was converted from its native format into PyTorch tensors. Min-max scaling was then applied, where each pixel value was divided by 255 (assuming a pixel intensity range of 0 to 255). This step ensured all pixel values resided within the range of 0 and 1. Following this, the data was normalized to the distribution observed within the training set. The normalization parameters, including the mean (0.2860) and standard deviation (0.3530), were determined empirically. This process standardized the data distribution, facilitating more efficient model training.

```

1 def train_test_dataframes(root: str) -> tuple[pd.DataFrame, pd.DataFrame]:
2     def dataset_to_dataframe(dataset: FashionMNIST) -> pd.DataFrame:
3         images: np.ndarray = dataset.data.flatten(1).numpy()
4         n_images, n_pixels = images.shape # Gather relevant dataset metadata
5
6         images = images.reshape(n_images, -1) # Flatten image arrays
7         column_names = [f"pixel_{i}" for i in range(n_pixels)]
8         data = pd.DataFrame(images, columns=column_names)
9         data["label"] = dataset.targets.numpy() # Append labels
10        return data
11
12    transform = v2.ToDtype(torch.float32, scale=True)
13    train = FashionMNIST(root, train=True, transform=transform, download=True)
14    test = FashionMNIST(root, train=False, transform=transform, download=True)
15    return dataset_to_dataframe(train), dataset_to_dataframe(test)

```

Figure 5: The method used to preprocess the PyTorch dataset into a Pandas DataFrame. The data is transformed from an image to a 28×28 float32 tensor and scaled appropriately.

Finally, the preprocessing pipeline incorporated random erasing, a technique introduced by Zhong et al. [71]. This augmentation strategy further promotes model regularization and generalization, leading to improved performance on unseen data.

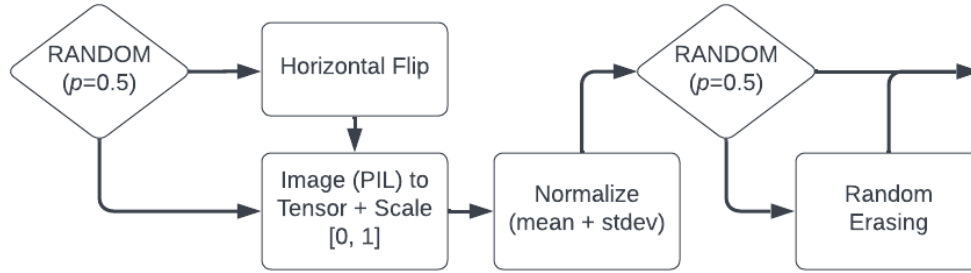


Figure 6: The augmentation pipeline employed during model training, aimed at artificially enhancing dataset diversity.

4.3 Model Development

The architecture of the CNN model was developed through an iterative design process. The initial design drew inspiration from the VGG architecture [54], but it was subsequently modified to accommodate the specific characteristics of the Fashion-MNIST dataset. Notably, the VGG architecture was originally designed for processing high-resolution images with three color channels and a resolution of 32×32 pixels. Given the lower resolution and single-channel nature of the Fashion-MNIST images, a simplified architecture was adopted to address computational efficiency concerns during model training.

The first iteration of the CNN architecture comprised two convolutional layers with ReLU activation functions, followed by a max pooling layer. The flattened output from this initial convolutional block was then fed into two fully connected (FC) layers for fashion classification.

The initial model achieved an accuracy of 88%. To improve performance, an iterative refinement process was undertaken. This process led to the incorporation of batch normalization within the convolutional block and the application of the previously described data augmentation pipeline. The final architecture consisted of two convolutional layers, each containing 64 neurons, followed by a max pooling layer. The flattened output from this convolutional block was then fed into a large FC layer with a dimension of 9216×1024 , followed by a final FC classification layer with a shape of 1024×10 . This configuration enhanced the model's ability to capture distinctive features within the Fashion-MNIST images, ultimately leading to improved classification accuracy.

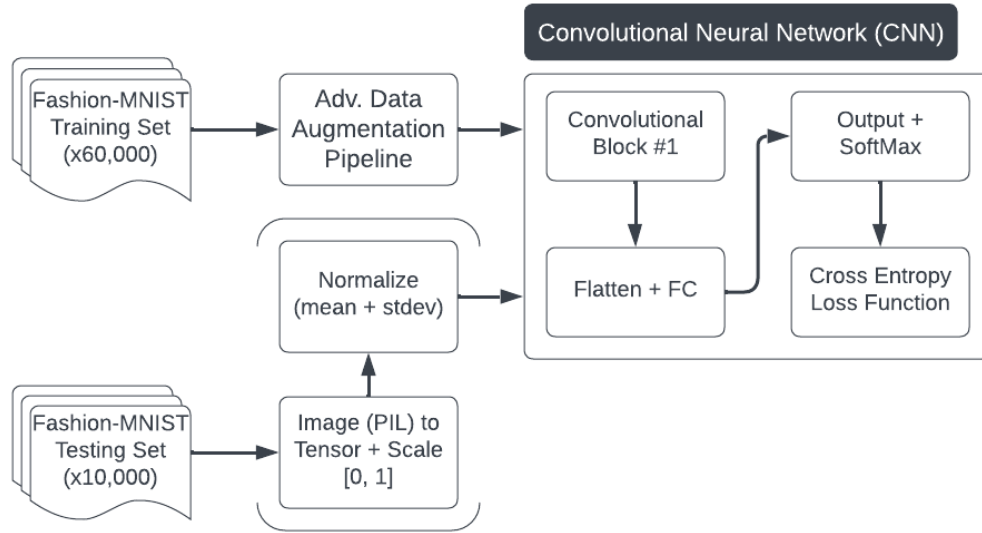


Figure 7: The basic architecture of the model in addition to the associated data augmentation techniques.

5 Experiments and Results

5.1 Introduction

In this section, we present the outcomes of our experiments on fashion classification using various machine learning models with the Fashion-MNIST dataset. We'll start by discussing the hardware and software setup, followed by an overview of the data preprocessing steps. Then, we'll evaluate the performance of each model, analyzing accuracy metrics and optimization results. Our exploration covers KNN, SVM, GNB, Random Forest, AdaBoost, Gradient Boost, FCANN, and CNN. We'll highlight the best-performing models, focusing on their accuracy, generalization ability, and environmental impact.

5.2 Experimental Setup

5.2.1 Hardware and Software Utilized

The experiments were conducted on a single laptop equipped with an Intel i7-11800H CPU @ 2.30 GHz, an NVIDIA GeForce RTX 3060 Laptop GPU, and 32 GB of DDR4 memory. This hardware configuration primarily aimed to accelerate model training and inference, without influencing the models' intrinsic predictive performance.

To manage the software environment, Windows Subsystem for Linux 2 (WSL2) was leveraged in conjunction with Mini-Conda to create a virtualized environment. This environment incorporated scikit-learn for traditional machine learning classification tasks and PyTorch for deep learning models, such as the FCANN and CNN.

Several popular Python deep learning frameworks, including TensorFlow and Keras, were initially evaluated for exploring fashion classification models on the Fashion-MNIST dataset. These frameworks are widely adopted in industry due to their efficiency and popularity. However, the final CNN architecture necessitated a dynamic computation graph, which PyTorch inherently supports through its runtime construction. Consequently, to ensure consistency and leverage this capability, PyTorch was chosen for implementing all deep learning classifiers within this study.

5.2.2 Data Preprocessing Pipelines

For machine learning models implemented using scikit-learn, the Fashion-MNIST dataset was preprocessed and stored as a Pandas DataFrame. This data structure facilitated efficient interaction between the models and scikit-learn's functionalities. While stored within the DataFrame, the data was normalized to a range of [0,1] using min-max scaling.

However, it is important to note that no advanced data augmentation techniques were employed to expand the dataset artificially.

In contrast, deep learning models leveraged PyTorch’s data loaders in conjunction with custom pipelines to achieve significant dataset diversification. These pipelines incorporated data augmentation techniques, enabling the models to capture more generalizable features during training. This process aimed to improve the robustness of the models and reduce the risk of overfitting the training data.

5.3 ML Models Used for Experimentation

5.3.1 K-Nearest Neighbors (KNN)

We investigated the efficacy of KNN for image classification on the Fashion-MNIST dataset. The data was preprocessed by flattening each image into a one-dimensional vector and subsequently converted into a DataFrame to facilitate integration with scikit-learn. Initial experimentation utilized the library’s default parameters, which include Euclidean distance as the similarity metric and a value of K (number of neighbors) set to 5. This baseline configuration achieved an accuracy of 85.54%.

To optimize the K-NN model, a comprehensive grid search was conducted encompassing 24 distinct configurations. The search explored variations in the number of neighbors (K), distance metrics (Euclidean and Manhattan), and weighting schemes (uniform and distance-based). Five-fold cross-validation was employed to evaluate each configuration. The optimal configuration, identified through the grid search, consisted of 8 neighbors, Manhattan distance, and distance-based weighting, achieving an accuracy of 86.04%.

Notably, computational resource limitations necessitated performing the five-fold cross-validation on a randomly sampled one-third subset of the original dataset. Utilizing the entire dataset for cross-validation was estimated to exceed two hours in total execution time, with each model configuration requiring approximately eight minutes for training. A detailed discussion regarding the environmental implications associated with this grid search is presented in Section 5.6: *Environmental Impact Analysis*.

5.3.2 Support Vector Machine (SVM)

Following the exploration of KNN, we evaluated the performance of SVMs for fashion classification. The data preprocessing steps mirrored those employed for KNN, involving image flattening and conversion to a Pandas DataFrame for compatibility with scikit-learn. It is well-documented that the SVM algorithm exhibits a computational complexity of $O(n_{features} * n_{observations}^2)$, rendering it less efficient compared to other supervised learning methods. To address this constraint, a subsample of 10,000 data points was utilized for both training and testing the model.

Initial experimentation adopted the default hyperparameters provided by scikit-learn, achieving an accuracy of 85.56%. Subsequently, a random search cross-validation approach was implemented to optimize hyperparameter selection. Random search offers greater flexibility in exploring a wider range of hyperparameter values, particularly those with continuous domains, compared to grid search. We evaluated 20 models with randomly chosen hyperparameters, but none surpassed the accuracy achieved with the default settings.

A confusion matrix is presented in Figure 8 to gain further insights into the model’s behavior. This visualization helps identify potential biases within the model and areas for improvement. Notably, the model struggles to differentiate between t-shirts, pullovers, and coats from traditional shirts. Conversely, performance in other classes is promising, with accuracy reaching as high as 96.8% for the bag class. It is important to acknowledge that due to computational limitations, the performance depicted in the confusion matrix may not be fully representative of a fully optimized SVM model.

5.4 Introduction

In this section, we present the outcomes of our experiments on fashion classification using various machine learning models with the Fashion-MNIST dataset. We’ll start by discussing the hardware and software setup, followed by an overview of the data preprocessing steps. Then, we’ll evaluate the performance of each model, analyzing accuracy metrics and optimization results. Our exploration covers KNN, SVM, GNB, AdaBoost, Gradient Boost, FCANN, and CNN. We’ll highlight the best-performing models, focusing on their accuracy, generalization ability, and environmental impact.

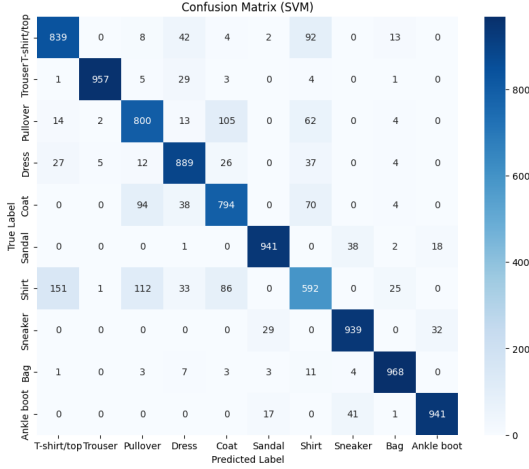


Figure 8: This confusion matrix depicts the performance of an SVM model trained on a subset of the Fashion-MNIST dataset. The rows represent the predicted labels, and the columns represent the ground truth. Each value represents the number of images incorrectly classified.

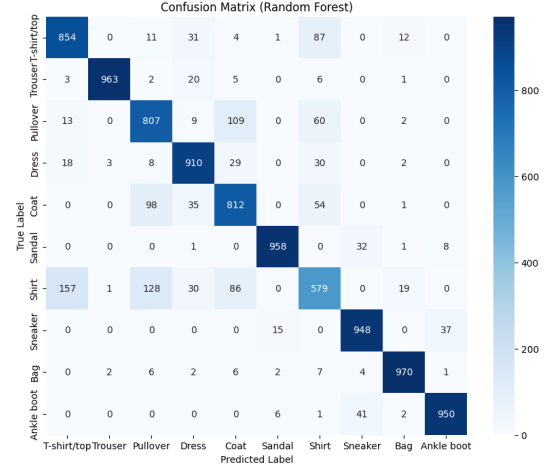


Figure 9: This confusion matrix visualizes the classification accuracy of a Random Forest model on the Fashion-MNIST dataset.

5.4.1 Gaussian Naive Bayes (GNB)

In addition to our investigation of various machine learning algorithms for fashion classification, we evaluated the performance of the GNB classifier. Utilizing scikit-learn’s implementation, the initial model achieved an accuracy of 58.56%. This accuracy fell short of the performance observed with previously explored algorithms, aligning with findings reported by Xiao et al. [68]. Given this lower baseline accuracy, further hyperparameter optimization and cross-validation were not pursued within the scope of this study. It is important to acknowledge that prior research has explored GNB on the Fashion-MNIST dataset [68, 42, 21]. Therefore, extensive optimization efforts for this specific algorithm were deemed outside the purview of this work, allowing us to prioritize the exploration of more promising avenues for fashion classification.

Beyond the observed performance limitations, the application of GNB to the Fashion-MNIST dataset is further challenged by inherent assumptions within the algorithm. Notably, GNB relies on the assumption of feature independence, which is violated in this context. Pixel values within an image exhibit a strong spatial correlation, meaning a single pixel’s intensity can significantly influence its neighbors. This is particularly relevant considering the preprocessing step that transforms detailed images into 28x28 grayscale representations. During scaling, the value of a single pixel in the original image can potentially contribute to the values of multiple pixels in the reduced representation, thus violating the independence assumption. Additionally, GNB is known for its sensitivity to noise. Given the inherent variability within fashion images (e.g., slight variations in clothing folds or wrinkles), the classifier is likely to struggle with accurate classifications on this dataset.

5.5 Report of Experimental Results

5.5.1 Random Forest

We implemented the Random Forest classifier to further our exploration of machine learning models and apply methods from our class. This was adapted to use each pixel as a feature. We did this through several preprocessing steps, where we flattened images from a 2D array to a 1D array. We also normalized the pixels to be in a 0 to 1 range. Through the use of 100 estimators, we were able to achieve an accuracy of 87.59%.

We also wanted to ensure that our model would perform well when employing k-fold cross-validation with a k value of 5, where we could see how accurate the model was given multiple subsets. The average accuracy was slightly lower than it was without using k-fold cross-validation, at 85%. While there is some variance between the results, the use of cross-validation gives us a conservative estimate as to the performance of the model with this method and prevents over-optimism that may occur from the evaluation of one test set.

Figure

5.5.2 AdaBoost

To complement the investigation into ensemble methods, we assessed the suitability of the AdaBoost algorithm. The implementation employed the SAMME.R variant, specifically designed for multi-class problems within scikit-learn. The initial evaluation, utilizing the default classifier on the training set and a separate testing set, yielded an accuracy of only 54.25%. Due to this significantly lower performance compared to other explored algorithms, further hyperparameter optimization for AdaBoost was deemed less promising. Consequently, efforts were prioritized towards optimizing alternative machine learning algorithms, particularly focusing on a novel CNN architecture, with the intent of identifying more efficient approaches for fashion classification.

Several factors might contribute to AdaBoost's lower performance compared to other ensemble methods on the Fashion-MNIST dataset. A prominent limitation is its sensitivity to noise within the training data. AdaBoost's core ensemble learning strategy involves iteratively focusing on "hard" examples – data points that are challenging to classify correctly. In the presence of noise, this approach can lead the model to overemphasize misleading or irrelevant examples, ultimately hindering the performance of the entire ensemble. Gradient boosting, the next algorithm explored, addresses this issue by prioritizing the minimization of overall training error across all data points. This focus on global error reduction leads to a more robust model and explains the superior performance of gradient boosting in this specific task.

5.5.3 Gradient Boost

Gradient Boosting was investigated as an alternative ensemble method for fashion classification. The initial model, evaluated using the entire dataset, achieved a promising accuracy of 89.64%. This result stands in stark contrast to the significantly lower performance observed with AdaBoost (54.25%). Given this initial success, a comprehensive grid search was conducted using cross-validation to optimize hyperparameters related to loss function, maximum tree depth, and minimum samples per leaf. Interestingly, the grid search identified the default scikit-learn model as optimal. However, another configuration with comparable accuracy was discovered. This alternative configuration utilized a maximum tree depth of 5 and a minimum of 6 samples per leaf, compared to the default settings of unlimited depth and 20 samples per leaf, respectively. This alternative model achieved an accuracy of 89.01% on the entire dataset. It is important to note that, due to computational limitations, the grid search was performed on a randomly sampled one-third subset of the data.

5.5.4 Fully Connected Artificial Neural Network (FCNN)

To establish a baseline performance benchmark for deep learning models on the Fashion-MNIST dataset, we implemented an FCNN with the ReLU activation function. The architecture consisted of two hidden layers, the first with 754 input units feeding into 128 neurons, and the second with 128 neurons. The final classification layer comprised 128 units corresponding to the 10 fashion categories. Notably, the model did not incorporate any regularization techniques (e.g., L1/L2, dropout, or data augmentation). Stochastic gradient descent (SGD) served as the optimizer with a learning rate (η) of 0.002.

We opted for a device-agnostic codebase leveraging NVIDIA's CUDA platform to promote computational efficiency and improve model throughput. This, in conjunction with PyTorch, facilitated the dynamic offloading of the model and data samples to the GPU, if available. This approach capitalizes on the hardware acceleration capabilities of GPUs, specifically the CUDA cores designed for machine learning tasks, to achieve enhanced parallelization during training and inference. Additionally, for platforms that supported the operations, the model was further optimized through quantization (single-precision floating point).

The FCNN, equipped with the above mentioned optimizations, was subjected to extensive testing, achieving an accuracy of 77%. To assess the model's robustness to random weight initialization, it was trained multiple times. These training runs consistently yielded convergence around 77% accuracy, with minor fluctuations of a few tenths of a percentage point.

5.5.5 Convolutional Neural Network (CNN)

Building upon the exploration of an FCNN, we implemented a CNN to investigate the effectiveness of convolutional and max pooling layers for fashion classification. The initial iteration comprised two convolutional layers followed by a ReLU activation layer. Max pooling layers were incorporated to capture higher-level features within the input images. Two fully connected layers subsequently served as the classification component. This baseline CNN, coupled with stochastic gradient descent (SGD), achieved an accuracy of 83

To refine the architecture, batch normalization was introduced immediately after the convolutional layers, aiming to accelerate model convergence. This modification yielded an accuracy of 88% after 10 training epochs. The optimizer

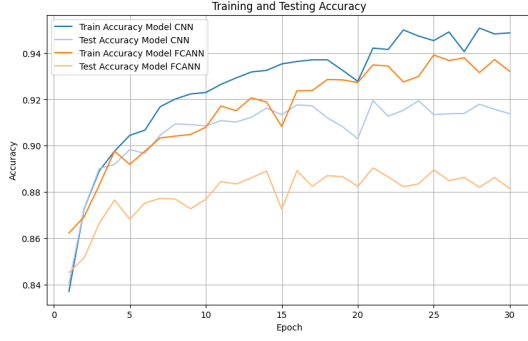


Figure 10: Training/Testing Accuracy (Epochs 1-30). CNN and FCANN accuracy on the first 30 epochs highlights learning trends and potential overfitting.

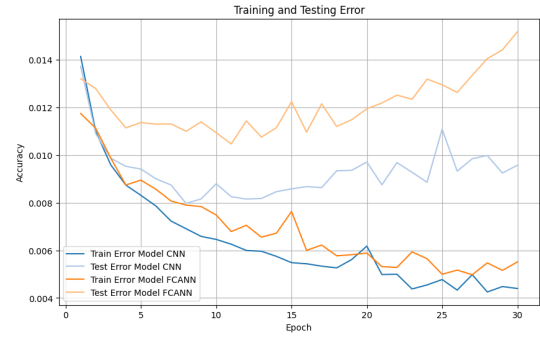


Figure 11: Training and Testing Error (Epochs 1-30). This figure complements Figure 10, visualizing the training and testing error of the CNN and FCANN models during the first 30 epochs. The lower error indicates better model performance.

was then transitioned to Adam, which offers advantages over SGD in the context of stochastic gradient descent. With Adam, the model’s accuracy climbed to 91%.

These advancements resulted in a robust model competitive with current SOTA models. However, to help model generalizability and prevent overfitting to the training/testing data, regularization techniques were employed. A data augmentation pipeline was implemented, introducing random image transformations such as horizontal flipping and erasing specific components. Additionally, dropout layers were strategically placed after each hidden layer to impede the network from developing an overreliance on any individual neuron. By incorporating these techniques and expanding the first convolutional layer to 64 neurons, the model’s accuracy improved to 92%. Further architectural enhancements were attempted, including replacing convolutional layers with various ensemble learners, but none surpassed this achieved accuracy.

Figures 10 and 11 illustrate the model’s accuracy and error rate throughout the training process. Several key observations can be gleaned from these figures. Firstly, the CNN architecture consistently achieved higher accuracy compared to the FCANN, highlighting the efficacy of convolutional blocks in extracting relevant features from image data. Secondly, the implementation of regularization techniques appears to enhance model robustness when compared to the FCANN. However, a degree of overfitting to the training data is still evident, suggesting potential for further improvement in model generalizability.

5.6 Environmental Impact Analysis

While environmental impact analysis has garnered increasing attention in machine learning, particularly regarding hyperparameter optimization, limitations exist in the current body of research focused on image classification. Many studies estimate search costs using generic metrics like "GPU days" without specifying the hardware, hindering generalizability. While Tanveer et al. [60] offer a table on the environmental impacts of training models on CIFAR-10 [39], their analysis relies on an Nvidia GTX 1080 GPU, a model for which modern carbon emission calculators lack specific metrics. To address this, Lacoste et al. [40] present the Machine Learning Impact Calculator, enabling more precise carbon footprint assessments based on specific hardware and regional electricity grids. Table 1 can be utilized to illustrate the carbon emissions associated with hyperparameter optimization for various models on Amazon Web Services in the us-east-2 region, considering its carbon efficiency and employing a more representative GPU, such as the RTX 2080.

Our work sheds light on the significant environmental benefits achievable through careful model design and optimization. Compared to SOTA models for image classification on Fashion-MNIST, our proposed convolutional neural network exhibits a substantially lower carbon footprint. We employed the codecarbon [47] library to comprehensively assess the carbon emissions associated with training our model. This library measures power consumption across various hardware components (e.g., GPU, CPU, RAM) during training, enabling an accurate estimation of total energy usage. Our model’s training process consumed 0.1506 kWh, translating to an estimated carbon footprint of 0.0858 kgCO₂. In contrast, the aforementioned SOTA models generate over 34 times the carbon emissions while achieving only a marginal improvement (4%) in accuracy on the Fashion-MNIST dataset.

Table 1: Carbon Emissions Comparison of Machine Learning Models on the CIFAR-10 Dataset

Architecture	Parameters (M)	Accuracy	Carbon Emitted (kgCO ₂ eq.)
AmoebaNet-B + cutout [53]	34.9	97.87%	9,264.78
NASNetA + cutout [72]	3.3	97.60%	5,882.40
PNAS [46]	3.2	96.59%	661.77
NAONet [49]	10.6	97.89%	588.24
SETN + cutout [18]	4.6	97.31%	5.30
SMASHv2 [12]	16	95.97%	4.41
Fine-Tuning DARTS [60]	3.9	97.52%	2.94

5.7 Detailed Results of the Best-Performing Model(s)

Looking at the results of all the techniques we employed, as shown in Table 2, CNNs had the greatest accuracy of up to 92% when employing batch normalization and optimization techniques, such as Adam. There is substantial generalization in the CNN model due to their layered structure and ability to identify different textures. Because of their depth with convolutional layers and pooling layers, they are also able to learn about the overall relationship between components in an image. Gradient boosting had the next highest accuracy of 89.64% due to the slight improvements in grid search optimization. It also showed strong generalization by building on weak learners, but careful consideration of managing parameters such as tree depth should continue to be monitored to stop overfitting from occurring. Random Forests followed with an accuracy of 87.59%. Random Forest is extraordinary at generalization because it can take the average of decision trees and lower the amount of variance that may occur. The next highest method was KNN, where we were able to enhance the parameters and use grid search for an 86.04% accuracy. Factors such as the number of neighbors and distance formula applied can impact its ability to generalize well, and because the variance and bias can change depending on the size of K, it shows that KNN is difficult to scale for image classification. SVM achieved 85.56% accuracy, but were unable to progress past the default settings, even with hyperparameter optimization through random search. They also have adequate generalization capabilities when using proper regularization techniques. Artificial Neural Networks can have acceptable generalization abilities if they are using acceptable regularization techniques, as shown in the 77% accuracy we achieved. This also shows that some overfitting may occur because of the limited data augmentation. Naive Bayes had a disappointing accuracy of 58.56%, which shows that it is poor at generalizing image datasets like Fashion-MNIST. This primarily occurred because of the need for features to be independent, which will not happen in complex image datasets as pixels are often correlated to each other. Finally, AdaBoost had the lowest accuracy of all the methods with 54.25%. Due to the nature of outliers and noisy data in image classification problems, this will lead to low generalization and possible overfitting.

Because CNNs are used most often in image classification tasks, the main objective of our experiment was to lower carbon emissions through minimal trade-offs in accuracy. With SOTA models achieving up to 9,264.78 kgCO₂, we needed to be able to create an environmentally friendly architecture that minimized a change in accuracy. By creating a model with 92% accuracy and only emitting 0.0858 kgCO₂, we have been successful in both of the achievements we sought.

Table 2: Accuracy and Hyperparameter Comparison of Machine Learning Models

Model	Accuracy	Model Hyperparameters
K-Nearest Neighbors	86.04%	$K = 8, p = 1, \text{weight} = \text{distance}$
Support Vector Machine	85.56%	$C = 1, \text{kernel} = \text{rbf}$
Gaussian Naive Bayes	58.56%	Default
Random Forest	87.59%	estimators = 100, criterion = <i>gini</i>
AdaBoost	54.25%	Default
Gradient Boost	89.64%	estimators = 100, learning rate = 0.1
FCANN	77%	SGD Optimizer, learning rate = 0.002
CNN	92%	Adam Optimizer, learning rate = 0.002

6 Discussion

6.1 Introduction

This discussion section analyzes and interprets the findings from our various machine learning models. Key considerations include a comparative assessment of model performance on the Fashion-MNIST dataset, alongside a detailed examination of the associated environmental impacts incurred during training. The section subsequently highlights the contributions of this research to the field of image classification. Finally, it acknowledges the limitations inherent to both the chosen dataset and the proposed models, while proposing potential avenues for improvement and future work.

6.2 Experimental Results and Analysis

6.2.1 Performance Comparison

Our investigation involved a comprehensive analysis of eight models, each evaluated under a wide range of hyperparameter configurations. This exploration provided valuable insights into the inherent complexities of the Fashion-MNIST dataset. Three models emerged as particularly noteworthy for this study.

The first model tested, KNN, achieved an accuracy of 86.04%. This result established a valuable baseline metric for assessing the predictive performance of subsequent models. Gradient Boosting, another prominent model, leverages gradient descent to minimize ensemble error during training. Consequently, this model achieved an accuracy of 89.64%, outperforming most contemporary FCNNs.

The best-performing model in our study employed a CNN architecture with various regularization techniques specifically designed to promote generalization during training. This CNN achieved an accuracy of 92% on the Fashion-MNIST dataset while maintaining a low environmental footprint. While this performance falls short of current SOTA models, such as Fine-tuned DARTS [60] with its reported accuracy of 96%, it demonstrates strong competitiveness.

6.2.2 Environmental Impact

While the proposed models achieved a level of accuracy slightly below that of SOTA models, they exhibited a significant advantage in terms of environmental impact compared to previously published work. Notably, the carbon footprint associated with training the Fine-tuned DARTS model [60], a representative SOTA approach, was over 34 times greater than our most performant model. This substantial difference highlights the environmental cost of complex models. Training such models often necessitate significant computational resources, leading to increased energy consumption and associated greenhouse gas emissions, particularly CO₂, contributing to global warming and climate change.

6.3 Contributions of This Work

6.3.1 Novelty

This study presents a novel image classification approach that prioritizes model performance and environmental impact. This is the first work to explicitly address this trade-off by establishing a Pareto frontier between predictive accuracy and the associated carbon footprint. We demonstrate this by utilizing the codecarbon library to monitor and minimize carbon emissions during model training. This approach yielded models with accuracy only 4% lower than SOTA CNN models while achieving a remarkable 34-fold reduction in carbon emissions compared to the same SOTA model. This significant reduction is attributed to our focus on sustainable model design, incorporating principles of comprehensive lifecycle management and energy-efficient algorithms. By prioritizing environmental responsibility alongside performance, this research advances the field of image classification while promoting sustainable practices within the machine learning community.

6.3.2 Insights Gained

This study yielded valuable insights into both machine learning algorithms and the environmental impact of model training. In terms of machine learning, limitations of certain models were identified. For example, GNB and AdaBoost exhibited significant limitations that restricted their accuracy on the Fashion-MNIST dataset. Conversely, other algorithms demonstrated superior performance and robustness for image classification tasks. Gradient Boosting and CNNs emerged as the top performers among the tested models. These findings corroborate previous observations by Xiao et al. [68].

Furthermore, the environmental impact analysis shed light on the significant carbon footprint associated with model training. This research extended the application of environmental impact analysis, previously focused on transformer

architectures, to the domain of image classification. This novel approach provided new insights into the environmental footprint of both established and proposed models.

6.4 Limitations

6.4.1 Data Limitations

The Fashion-MNIST dataset, while widely used in computer vision research, possesses limitations that can hinder model generalizability. A primary concern is the dataset’s standardized, low-resolution (28×28 pixels) grayscale format. This format significantly reduces the information content of images compared to real-world scenarios, where data often exhibits higher resolution and richer color palettes. Consequently, models trained solely on Fashion-MNIST may not fully exploit their feature learning capabilities due to the limited complexity of the training data.

Furthermore, the dataset lacks inherent variability in terms of image orientation, scale, and background noise. Real-world images often exhibit diverse characteristics in these aspects. The absence of such variations likely introduced through aggressive pre-processing techniques, limits the dataset’s true diversity and introduces a degree of artificiality. This can negatively impact model robustness, as models trained on the simplified Fashion-MNIST distribution may struggle to generalize to real-world data with its inherent complexities.

6.4.2 Model Limitations

This study identified limitations associated with applying deep learning architectures like FCANNs and CNNs to the Fashion-MNIST dataset. One key challenge is interpretability, particularly for feature-dense datasets. In Fashion-MNIST, each data point comprises 784 discrete features, each with 256 possible values. This high dimensionality makes it difficult to visually understand the features influencing the models’ inferences.

Another limitation observed in this study was overfitting, as evidenced by the results presented in Figures 10 and 11. The employed regularization techniques may not have been entirely sufficient to prevent overfitting, potentially hindering the models’ ability to generalize to unseen data.

Scalability is an additional consideration, particularly for the proposed CNN architecture. The current model was trained on a predefined dataset with no inherent mechanism for continuous learning. This limitation could restrict its applicability in real-world scenarios, especially within the dynamic fashion industry. Section 6.5 will explore potential solutions to address this scalability challenge.

6.5 Future Works

6.5.1 Model Refinement

Despite achieving comparable performance to state-of-the-art (SOTA) models on the Fashion-MNIST dataset, several avenues exist for further improvement of the proposed CNN architecture.

- Optimizing the learning rate schedule could potentially enhance the model’s ability to minimize the loss function. Li and Arora [43] proposed a framework specifically designed for this purpose. However, exploring alternative approaches, such as dynamic batch size adjustments [59], could also be beneficial.
- Employing more robust regularization techniques could further improve model generalizability. This would allow for training over a greater number of epochs, potentially facilitating full convergence and reducing the risk of overfitting observed in the current model (refer to Figures 10 and 11).
- Incorporating a more comprehensive data augmentation pipeline presents another opportunity for model improvement. Recent advancements in data augmentation techniques, as comprehensively surveyed by Shorten and Khoshgoftaar [57], offer promising avenues for artificially expanding the dataset size. This could force the model to learn more generalizable features, ultimately leading to stronger performance on unseen data.

6.5.2 Exploration of Additional Models

Beyond the investigated machine learning models, exploring alternative architectures holds promise for further optimizing performance on the Fashion-MNIST dataset. Notably, the transformer architecture [63], recently applied to image classification by Dosovitskiy et al. [20], achieved SOTA accuracy on the CIFAR-10 dataset (99.5%). This success is attributed in part to the transformer’s unique attention mechanism, which can capture long-range dependencies more effectively than previously established architectures. Consequently, transformer models have the potential to outperform CNN in terms of accuracy.

However, it is important to acknowledge the trade-off between performance and environmental impact. Transformer models often require a significantly higher number of trainable parameters compared to CNNs. This increase in complexity translates to a larger carbon footprint during the training process.

6.5.3 Environmental Sustainability

Environmental sustainability remains an under-explored aspect of machine learning research. While the field primarily focuses on model performance, a growing body of research acknowledges the significant environmental impact associated with extensive training runs, particularly for complex models. Recent studies, such as the work by Fournier et al. [22] exploring transformer model efficiency, demonstrate the potential for reducing computational overhead and consequently, carbon emissions. These findings suggest that techniques employed for transformer model distillation, such as parameter reduction, can apply to convolutional neural networks (CNNs) as well. By strategically reducing the number of trainable parameters, CNNs could achieve comparable accuracy with a smaller environmental footprint.

Furthermore, exploring alternative energy sources for powering machine learning training holds promise. Renewable energy sources such as wind, solar, and nuclear power offer a more sustainable alternative to traditional energy sources, as highlighted by Chu and Majumdar [15]. Transitioning to renewable energy infrastructure for large-scale training could significantly mitigate the environmental impact of machine learning. However, achieving widespread adoption of such sustainable practices necessitates a large-scale shift within the field.

7 Conclusion

This study investigated the trade-off between predictive performance and environmental impact in machine learning, focusing on the Fashion-MNIST dataset as a representative case study. By exploring the Pareto frontier within this domain, the research underscores the importance of environmentally conscious approaches to model development. Leveraging a widely used dataset, the study conducted a comprehensive review of existing literature to identify successful models while minimizing their associated carbon footprint.

Following the literature review, the research methodology centered on developing an environmentally friendly CNN architecture. This architecture achieved comparable performance to current SOTA models while exhibiting a significant 34-fold reduction in carbon emissions. The study further explored the impact of model selection by evaluating a range of machine learning algorithms. A thorough analysis of the results identified limitations inherent to both the chosen dataset and the proposed models. These limitations were subsequently addressed, and potential avenues for future research were outlined to further optimize the Pareto frontier within the context of environmentally sustainable machine learning.

References

- [1] ABDI, H., AND WILLIAMS, L. J. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics* 2, 4 (2010), 433–459.
- [2] ABUHAMAD, M., ABUHMED, T., MOHAISEN, A., AND NYANG, D. Large-scale and language-oblivious code authorship identification. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (2018), pp. 101–114.
- [3] ABUHAMAD, M., ABUHMED, T., MOHAISEN, D., AND NYANG, D. Large-scale and robust code authorship identification with deep feature learning. *ACM Transactions on Privacy and Security (TOPS)* 24, 4 (2021), 1–35.
- [4] AGARAP, A. F. Deep learning using rectified linear units (ReLU). *arXiv preprint arXiv:1803.08375* (2018).
- [5] AWAD, M., KHANNA, R., AWAD, M., AND KHANNA, R. Support vector machines for classification. *Efficient learning machines: Theories, concepts, and applications for engineers and system designers* (2015), 39–66.
- [6] BERGSTRA, J., AND BENGIO, Y. Random search for hyper-parameter optimization. *Journal of machine learning research* 13, 2 (2012).
- [7] BIAU, G., AND SCORNET, E. A random forest guided tour. *Test* 25 (2016), 197–227.
- [8] BLUMER, A., EHRENFEUCHT, A., HAUSSLER, D., AND WARMUTH, M. K. Occam’s razor. *Information processing letters* 24, 6 (1987), 377–380.
- [9] BOSCH, A., ZISSERMAN, A., AND MUNOZ, X. Image classification using random forests and ferns. In *2007 IEEE 11th international conference on computer vision* (2007), Ieee, pp. 1–8.

- [10] BOSSARD, L., GUILLAUMIN, M., AND VAN GOOL, L. Food-101—mining discriminative components with random forests. In *Computer Vision—ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part VI 13* (2014), Springer, pp. 446–461.
- [11] BRADY, M. Artificial intelligence and robotics. *Artificial intelligence* 26, 1 (1985), 79–121.
- [12] BROCK, A., LIM, T., RITCHIE, J. M., AND WESTON, N. Smash: One-shot model architecture search through hypernetworks, 2017.
- [13] CHAKRABORTY, S., HOQUE, M. S., RAHMAN JEEM, N., BISWAS, M. C., BARDHAN, D., AND LOBATON, E. Fashion recommendation systems, models and methods: A review. In *Informatics* (2021), vol. 8, MDPI, p. 49.
- [14] CHANDRA, M. A., AND BEDI, S. Survey on svm and their application in image classification. *International Journal of Information Technology* 13, 5 (2021), 1–11.
- [15] CHU, S., AND MAJUMDAR, A. Opportunities and challenges for a sustainable energy future. *nature* 488, 7411 (2012), 294–303.
- [16] CUNNINGHAM, P., AND DELANY, S. J. k-nearest neighbor classifiers: (with python examples). *arXiv preprint arXiv:2004.04523* (2020).
- [17] DENG, J., DONG, W., SOCHER, R., LI, L.-J., LI, K., AND FEI-FEI, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition* (2009), IEEE, pp. 248–255.
- [18] DONG, X., AND YANG, Y. One-shot neural architecture search via self-evaluated template network. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2019), pp. 3681–3690.
- [19] DONG, X., YU, Z., CAO, W., SHI, Y., AND MA, Q. A survey on ensemble learning. *Frontiers of Computer Science* 14 (2020), 241–258.
- [20] DOSOVITSKIY, A., BEYER, L., KOLESNIKOV, A., WEISSENBORN, D., ZHAI, X., UNTERTHINER, T., DEHGHANI, M., MINDERER, M., HEIGOLD, G., GELLY, S., ET AL. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [21] EDWARDS, C., AND YEN, A. Fashion-mnist.
- [22] FOURNIER, Q., CARON, G. M., AND ALOISE, D. A practical survey on faster and lighter transformers. *ACM Computing Surveys* 55, 14s (2023), 1–40.
- [23] FREUND, Y., AND SCHAPIRE, R. E. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences* 55, 1 (1997), 119–139.
- [24] FUKUSHIMA, K. Visual feature extraction by a multilayered network of analog threshold elements. *IEEE Transactions on Systems Science and Cybernetics* 5, 4 (1969), 322–333.
- [25] FUKUSHIMA, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological cybernetics* 36, 4 (1980), 193–202.
- [26] GAO, B., AND PAVEL, L. On the properties of the softmax function with application in game theory and reinforcement learning. *arXiv preprint arXiv:1704.00805* (2017).
- [27] GOODELL, J. W., KUMAR, S., LIM, W. M., AND PATTAIAK, D. Artificial intelligence and machine learning in finance: Identifying foundations, themes, and research clusters from bibliometric analysis. *Journal of Behavioral and Experimental Finance* 32 (2021), 100577.
- [28] GRANDINI, M., BAGLI, E., AND VISANI, G. Metrics for multi-class classification: an overview. *arXiv preprint arXiv:2008.05756* (2020).
- [29] GUO, G., WANG, H., BELL, D., BI, Y., AND GREER, K. Knn model-based approach in classification. In *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3-7, 2003. Proceedings* (2003), Springer, pp. 986–996.
- [30] HARRIS, E., MARCU, A., PAINTER, M., NIRANJAN, M., PRÜGEL-BENNETT, A., AND HARE, J. Fmix: Enhancing mixed sample data augmentation. *arXiv preprint arXiv:2002.12047* (2020).
- [31] HASTIE, T., ROSSET, S., ZHU, J., AND ZOU, H. Multi-class adaboost. *Statistics and its Interface* 2, 3 (2009), 349–360.
- [32] HE, K., ZHANG, X., REN, S., AND SUN, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016), pp. 770–778.
- [33] HE, K., ZHANG, X., REN, S., AND SUN, J. Identity mappings in deep residual networks. *arXiv preprint arXiv:1603.05027* (2016).

- [34] HUTTER, F., KOTTHOFF, L., AND VANSCHOREN, J. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [35] IOFFE, S., AND SZEGEDY, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (2015), pmlr, pp. 448–456.
- [36] KADAM, S. S., ADAMUTHE, A. C., AND PATIL, A. B. Cnn model for image classification on mnist and fashion-mnist dataset. *Journal of scientific research* 64, 2 (2020), 374–384.
- [37] KINGMA, D. P., AND BA, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [38] KOLCHINSKY, A., TRACEY, B. D., AND WOLPERT, D. H. Nonlinear information bottleneck. *Entropy* 21, 12 (2019), 1181.
- [39] KRIZHEVSKY, A., HINTON, G., ET AL. Learning multiple layers of features from tiny images.
- [40] LACOSTE, A., LUCCIONI, A., SCHMIDT, V., AND DANDRES, T. Quantifying the carbon emissions of machine learning. *arXiv preprint arXiv:1910.09700* (2019).
- [41] LECUN, Y., CORTES, C., AND BURGESS, C. The MNIST database of handwritten digits.
- [42] LEITHARDT, V. Classifying garments from fashion-mnist dataset through cnns. *Advances in Science, Technology and Engineering Systems Journal* 6, 1 (2021), 989–994.
- [43] LI, Z., AND ARORA, S. An exponential learning rate schedule for deep learning. *arXiv preprint arXiv:1910.07454* (2019).
- [44] LIAO, Z., AND COUILLET, R. A large dimensional analysis of least squares support vector machines. *IEEE Transactions on Signal Processing* 67, 4 (2019), 1065–1074.
- [45] LIN, D., TALATHI, S., AND ANNAPUREDDY, S. Fixed point quantization of deep convolutional networks. In *International conference on machine learning* (2016), PMLR, pp. 2849–2858.
- [46] LIU, C., ZOPH, B., NEUMANN, M., SHLENS, J., HUA, W., LI, L.-J., FEI-FEI, L., YUILLE, A., HUANG, J., AND MURPHY, K. Progressive neural architecture search, 2018.
- [47] LOTTICK, K., SUSAI, S., FRIEDLER, S. A., AND WILSON, J. P. Energy usage reports: Environmental awareness as part of algorithmic accountability. *arXiv preprint arXiv:1911.08354* (2019).
- [48] LU, D., AND WENG, Q. A survey of image classification methods and techniques for improving classification performance. *International Journal of Remote sensing* 28, 5 (2007), 823–870.
- [49] LUO, R., TIAN, F., QIN, T., CHEN, E., AND LIU, T.-Y. Neural architecture optimization, 2019.
- [50] NATEKIN, A., AND KNOLL, A. Gradient boosting machines, a tutorial. *Frontiers in neurorobotics* 7 (2013), 21.
- [51] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., DESMAISON, A., KÖPF, A., YANG, E., DEVITO, Z., RAISON, M., TEJANI, A., CHILAMKURTHY, S., STEINER, B., FANG, L., BAI, J., AND CHINTALA, S. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [52] PRAVEEN, U., FARNAZ, G., AND HATIM, G. Inventory management and cost reduction of supply chain processes using ai based time-series forecasting and ann modeling. *Procedia Manufacturing* 38 (2019), 256–263.
- [53] REAL, E., AGGARWAL, A., HUANG, Y., AND LE, Q. V. Regularized evolution for image classifier architecture search, 2019.
- [54] SENGUPTA, A., YE, Y., WANG, R., LIU, C., AND ROY, K. Going deeper in spiking neural networks: Vgg and residual architectures. *Frontiers in neuroscience* 13 (2019), 95.
- [55] SHAHEEN, M. Y. Applications of artificial intelligence (ai) in healthcare: A review. *ScienceOpen Preprints* (2021).
- [56] SHI, M., AND LEWIS, V. D. Using artificial intelligence to analyze fashion trends. *arXiv preprint arXiv:2005.00986* (2020).
- [57] SHORTEN, C., AND KHOSHGOFTAAR, T. M. A survey on image data augmentation for deep learning. *Journal of big data* 6, 1 (2019), 1–48.
- [58] SILVA, E. S., HASSANI, H., MADSEN, D. Ø., AND GEE, L. Googling fashion: forecasting fashion consumer behaviour using google trends. *Social Sciences* 8, 4 (2019), 111.
- [59] SMITH, S. L., KINDERMANS, P.-J., YING, C., AND LE, Q. V. Don’t decay the learning rate, increase the batch size. *arXiv preprint arXiv:1711.00489* (2017).

- [60] TANVEER, M. S., KHAN, M. U. K., AND KYUNG, C.-M. Fine-tuning darts for image classification. In *2020 25th International Conference on Pattern Recognition (ICPR)* (2021), IEEE, pp. 4789–4796.
- [61] TOMMASI, T., PATRICIA, N., CAPUTO, B., AND TUYTELAARS, T. A deeper look at dataset bias. *Domain adaptation in computer vision applications* (2017), 37–55.
- [62] VAN ENGELEN, J. E., AND HOOS, H. H. A survey on semi-supervised learning. *Machine learning* 109, 2 (2020), 373–440.
- [63] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, Ł., AND POLOSUKHIN, I. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [64] VERMA, S., SHARMA, R., DEB, S., AND MAITRA, D. Artificial intelligence in marketing: Systematic review and future research direction. *International Journal of Information Management Data Insights* 1, 1 (2021), 100002.
- [65] VIDAURRE, D., BIELZA, C., AND LARRANAGA, P. A survey of l1 regression. *International Statistical Review* 81, 3 (2013), 361–387.
- [66] WICKRAMASINGHE, I., AND KALUTARAGE, H. Naive bayes: applications, variations and vulnerabilities: a review of literature with code snippets for implementation. *Soft Computing* 25, 3 (2021), 2277–2293.
- [67] WONG, T.-T., AND YEH, P.-Y. Reliable accuracy estimates from k-fold cross validation. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1586–1594.
- [68] XIAO, H., RASUL, K., AND VOLLGRAF, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, 2017.
- [69] XUE, D., ZHOU, X., LI, C., YAO, Y., RAHAMAN, M. M., ZHANG, J., CHEN, H., ZHANG, J., QI, S., AND SUN, H. An application of transfer learning and ensemble learning techniques for cervical histopathology image classification. *IEEE Access* 8 (2020), 104603–104618.
- [70] ZAGORUYKO, S., AND KOMODAKIS, N. Wide residual networks. *arXiv preprint arXiv:1605.07146* (2016).
- [71] ZHONG, Z., ZHENG, L., KANG, G., LI, S., AND YANG, Y. Random erasing data augmentation. In *Proceedings of the AAAI conference on artificial intelligence* (2020), vol. 34, pp. 13001–13008.
- [72] ZOPH, B., VASUDEVAN, V., SHLENS, J., AND LE, Q. V. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (2018), pp. 8697–8710.